

RNNs & Transformers

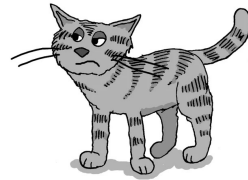
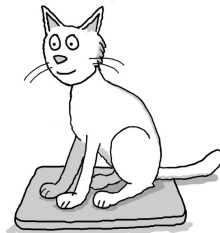
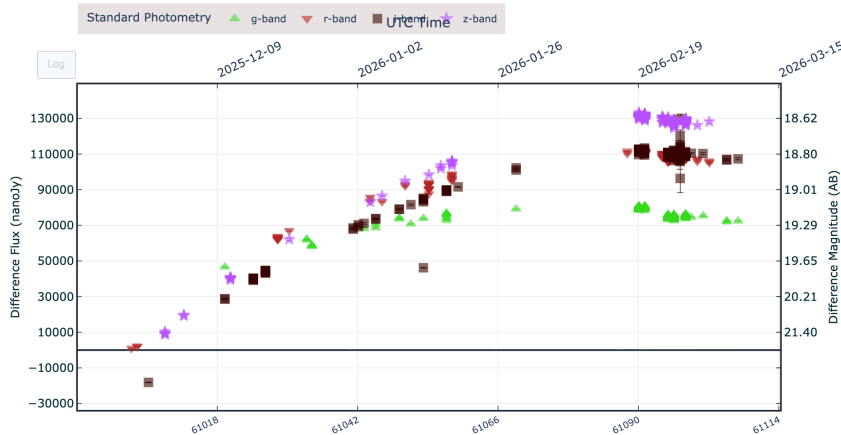
Modeling sequences, from light curves to language

Ioana Ciucă (Stanford/SLAC/UniverseTBD) · LSST-DA Data Science Fellowship Program

Why sequences, why now

01 / 20

- Rubin has issues its first ~800,000 scientific alerts in February 2026.
- The stream will reach up to about 7 million alerts per night during the 10-year LSST survey.
- A light curve is a sequence: irregular sampling, variable length, multiple bands.
- A sentence is a sequence too, pointing to a similar kind of mathematical object.
- **Question for today: how should a model use the past to figure out what comes next?**



"The cat sat on the mat is not a story. The cat sat on the other cat's mat is a story."

- John Le Carre

- 1980s — recurrent neural networks (RNNs): process a sequence by carrying context forward
- 1997 — long short-term memory (LSTM): gates make memory more stable.
- 2013 — word2vec: meaning as geometry in a learned vector space.
- 2014 — attention in neural machine translation.
- 2017 — the transformer: attention without recurrence.

**If the task was always sequence prediction,
why did capability suddenly explode?**

Key references: Rumelhart et al. 1986; Elman 1990; Hochreiter & Schmidhuber 1997; Mikolov et al. 2013; Bahdanau et al. 2014; Vaswani et al. 2017, Brown et al. 2020

What you should be able to do by the end

03 / 20

- Explain how tokens become vectors, and why static embeddings are not enough.
- Understand an RNN as a long computational graph and a transformer as a short one.
- Derive attention from the Nadaraya–Watson estimator: weighted averaging with a learned similarity.
- Sketch how information moves in a transformer block.
- Recognize why this matters for time-domain astronomy.
- Have fun!

- Split the input text into tokens coming from a fixed vocabulary.

- word

“the cat is running” → *[the] [cat] [is] [running]*

- sub-word

“the cat is running” → *[the] [cat] [is] [run] [##ing]*

- character

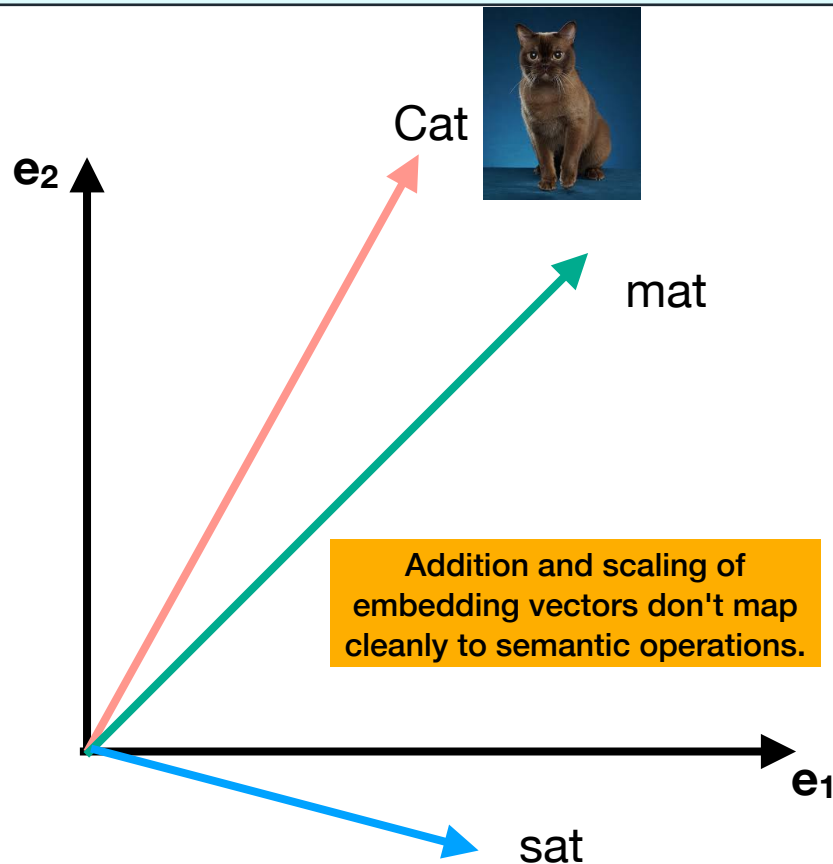
“the cat is running” → *[t] [h] [e] [c] [a] [t] ...*

- Out-of-vocabulary (OOV) means “not in the vocabulary”. Sub-words reduce this problem.
- Each token becomes an integer id, then a one-hot vector.

From tokens to vectors: embeddings

05 / 20

- One-hot makes every token orthogonal. There is no shared meaning
- A learned embedding (e.g. word2vec) places tokens in $\mathbb{R}^d$ where similarity becomes geometric closeness.
- Q: What is a possible limitation here?
- Lookup embedding is static. One vector for “cat” regardless of sentence.



$$P(t_1, \dots, t_n) = \prod_{i=1}^n P(t_i \mid t_1, \dots, t_{i-1})$$

- Sequence modeling breaks a sequence into repeated next-step predictions.
- The task is one conditional prediction, repeated many times: $P(\text{next} \mid \text{context})$.
- Formally, each token is predicted from its prefix: $P(t_i \mid t_1, \dots, t_{i-1})$.
- During **training**, we reward the model for putting high probability on the token that actually came next.

$$\mathcal{L} = - \sum_{i=1}^n \log P(t_i \mid t_1, \dots, t_{i-1})$$

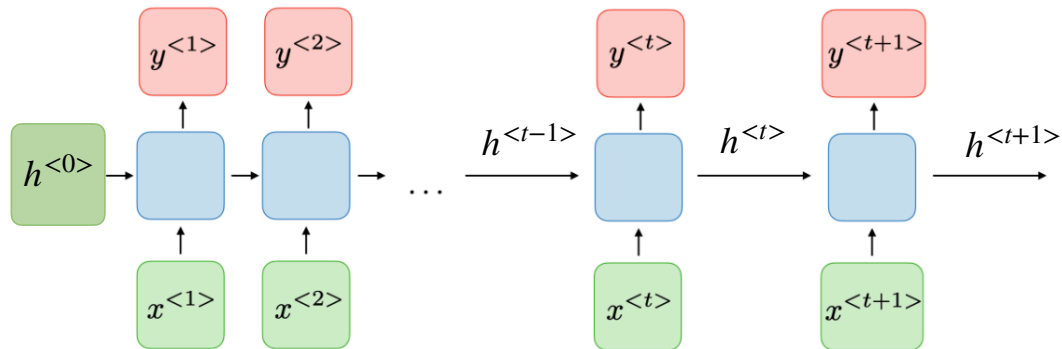
low probability on the observed next token $\rightarrow$ high loss

- What information from the past should the model carry forward?

The recurrent network: a long computational graph

07 / 20

- Carry a running summary along the sequence, which is the hidden state.
- Each token's representation now depends on what came before. The token is aware of the context.
- The sharing of parameters happens across time.
- But information and gradients travel through a long chain.



$$h_t = \phi(W_{hh}h_{t-1} + W_{xh}x_t + b_h), \quad y_t = W_{hy}h_t + b_y \quad (\text{same weights every step})$$

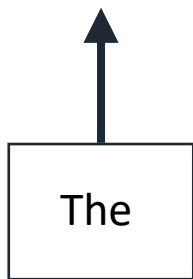
The recurrent network: a long computational graph

08 / 20

The

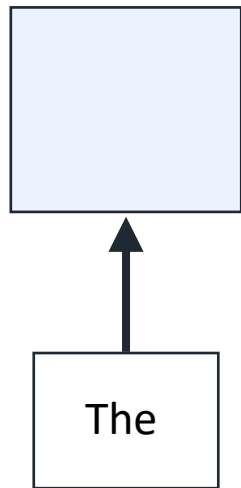
The recurrent network: a long computational graph

08 / 20



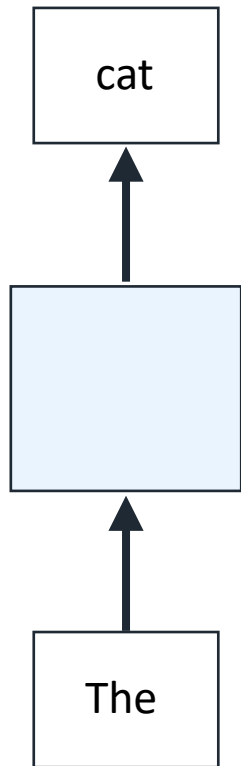
The recurrent network: a long computational graph

08 / 20



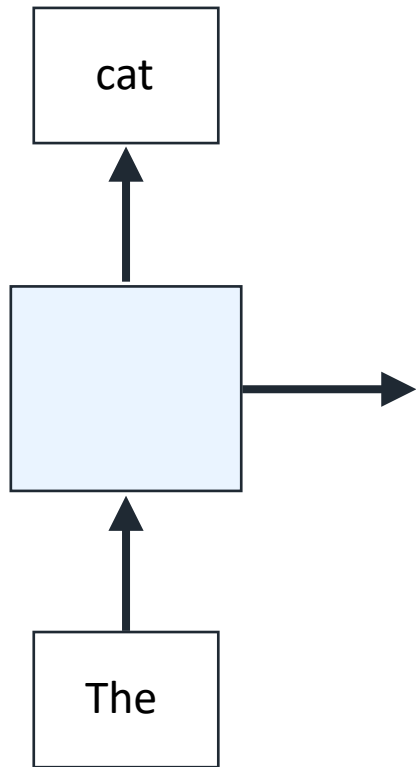
The recurrent network: a long computational graph

08 / 20



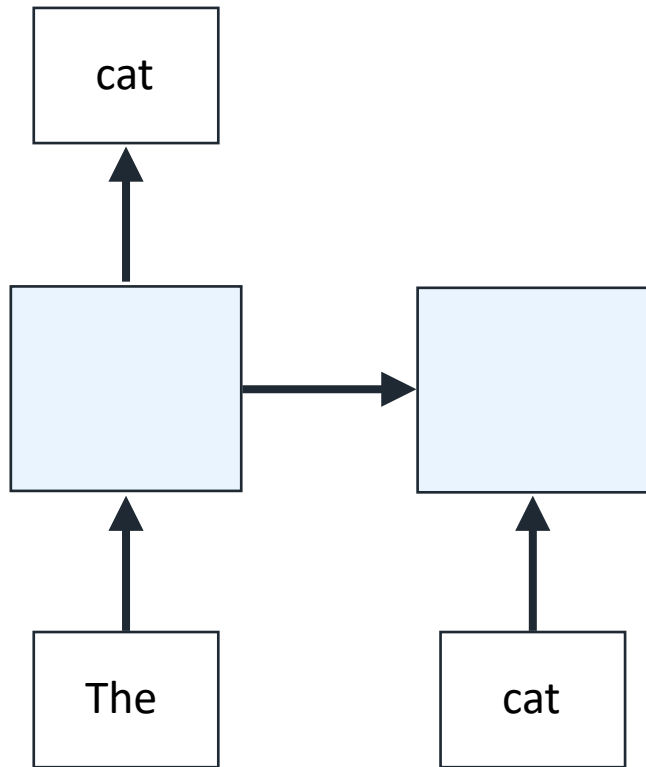
The recurrent network: a long computational graph

08 / 20



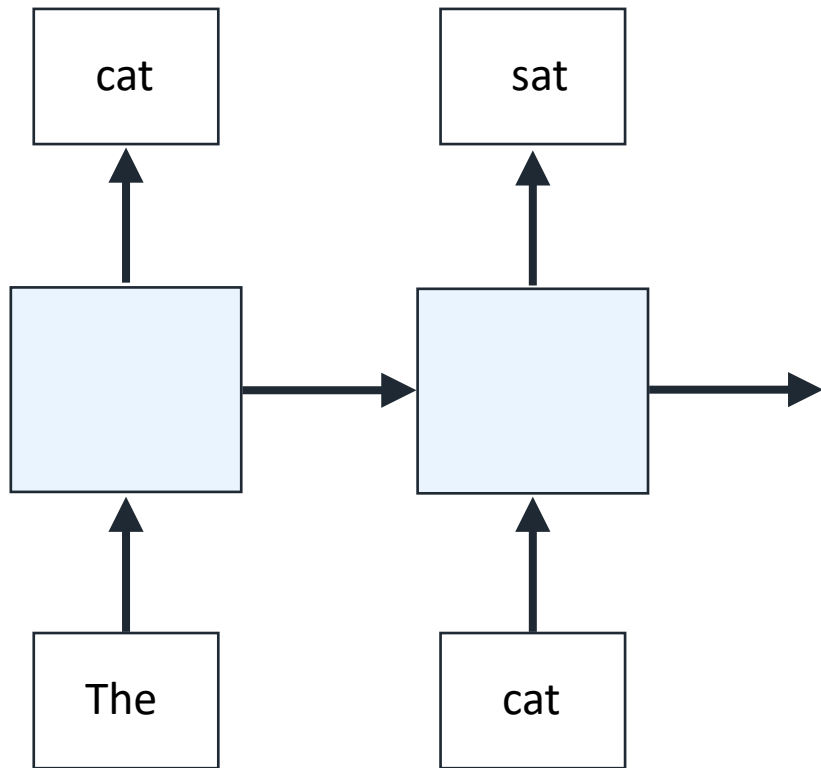
The recurrent network: a long computational graph

08 / 20



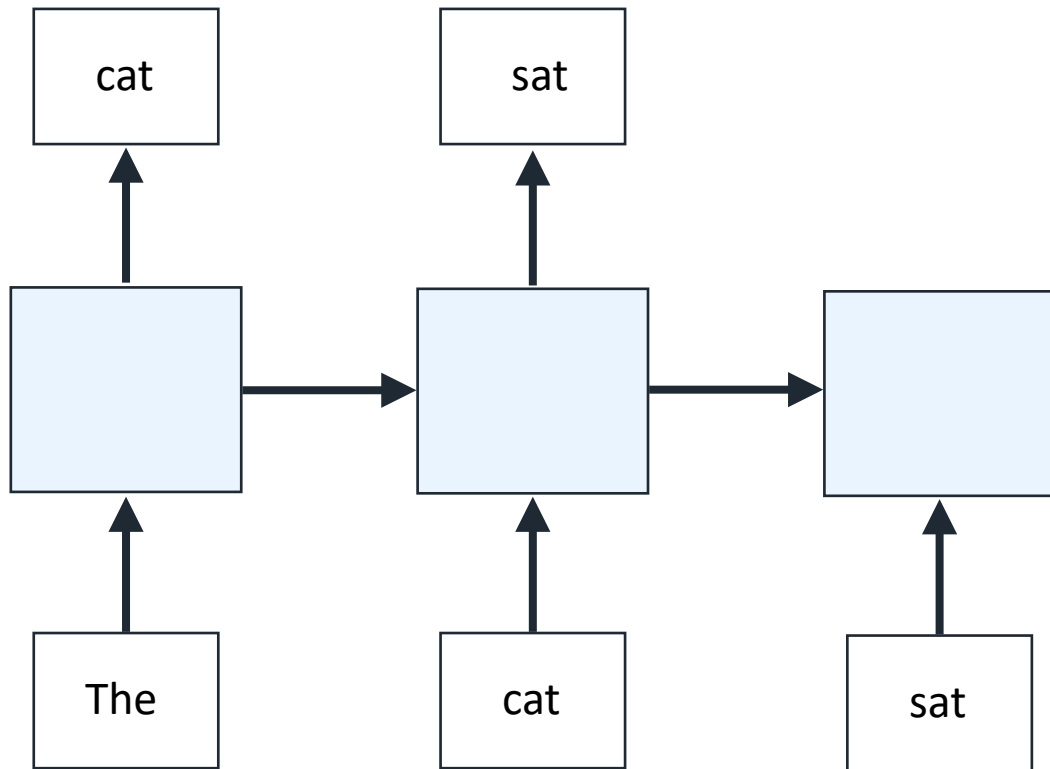
The recurrent network: a long computational graph

08 / 20



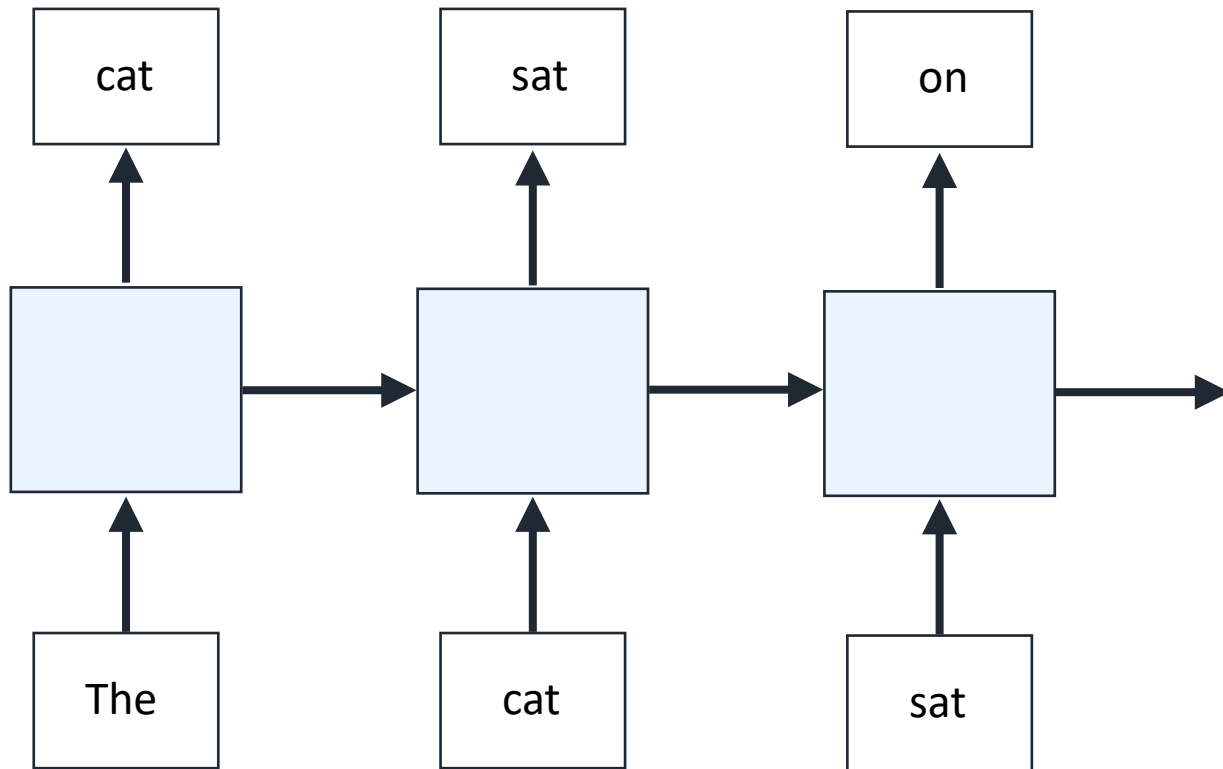
The recurrent network: a long computational graph

08 / 20



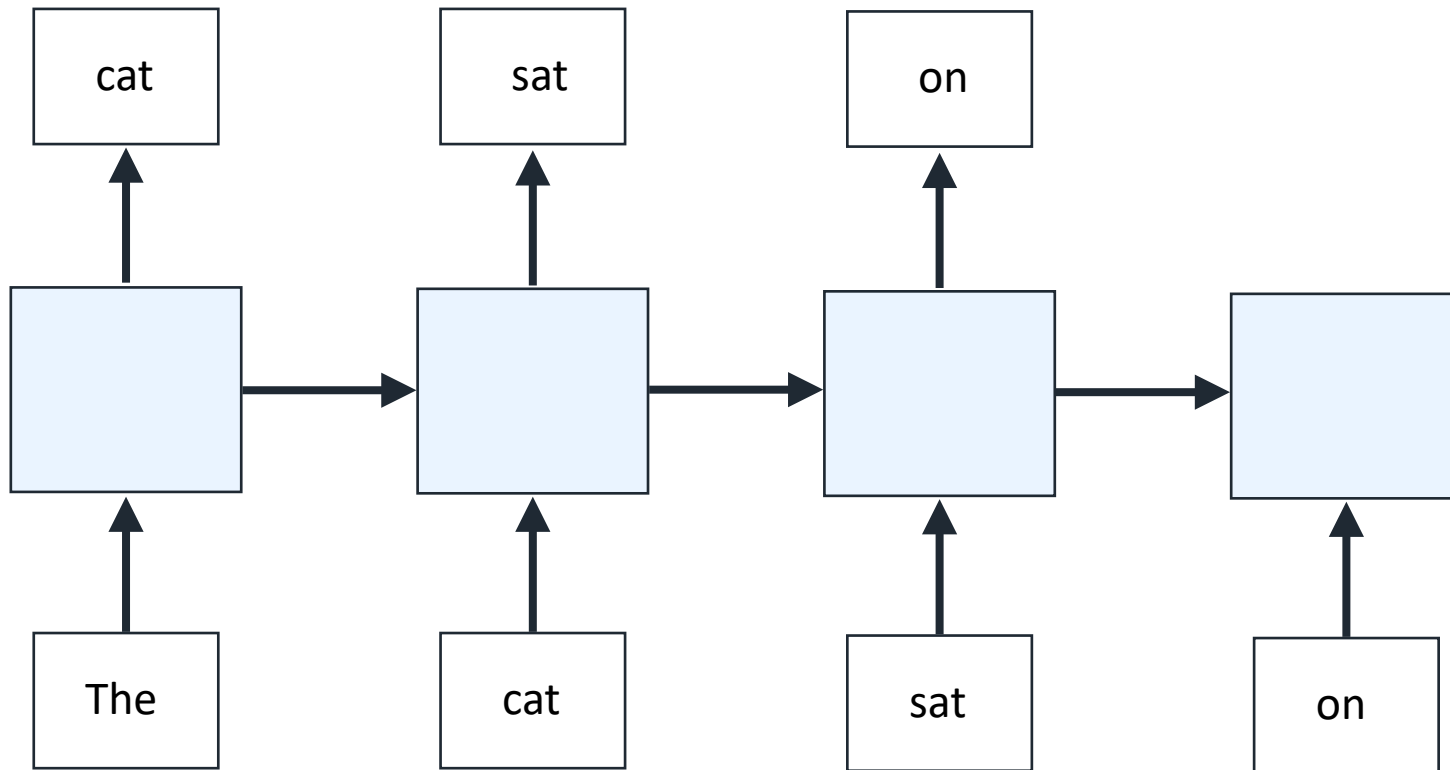
The recurrent network: a long computational graph

08 / 20



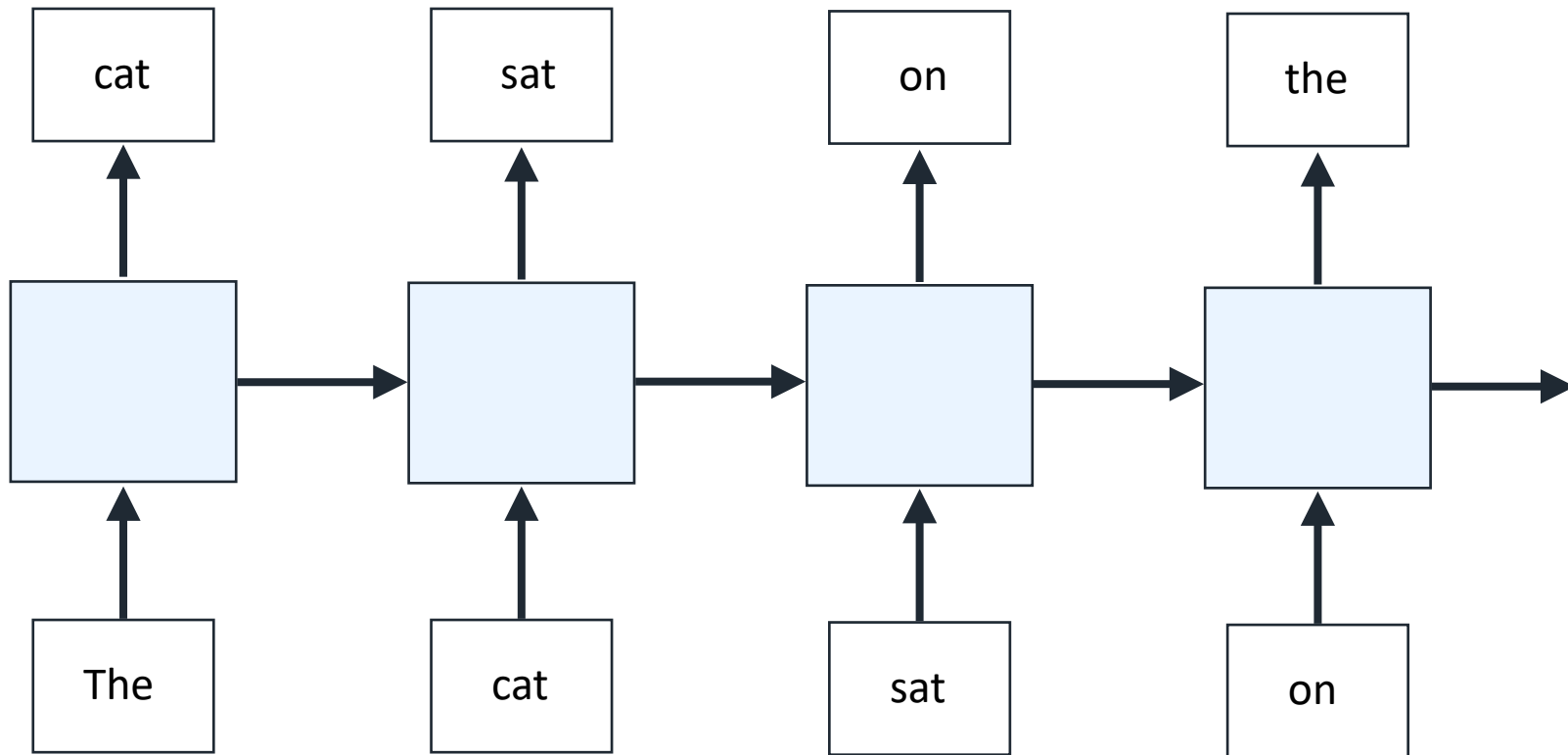
The recurrent network: a long computational graph

08 / 20



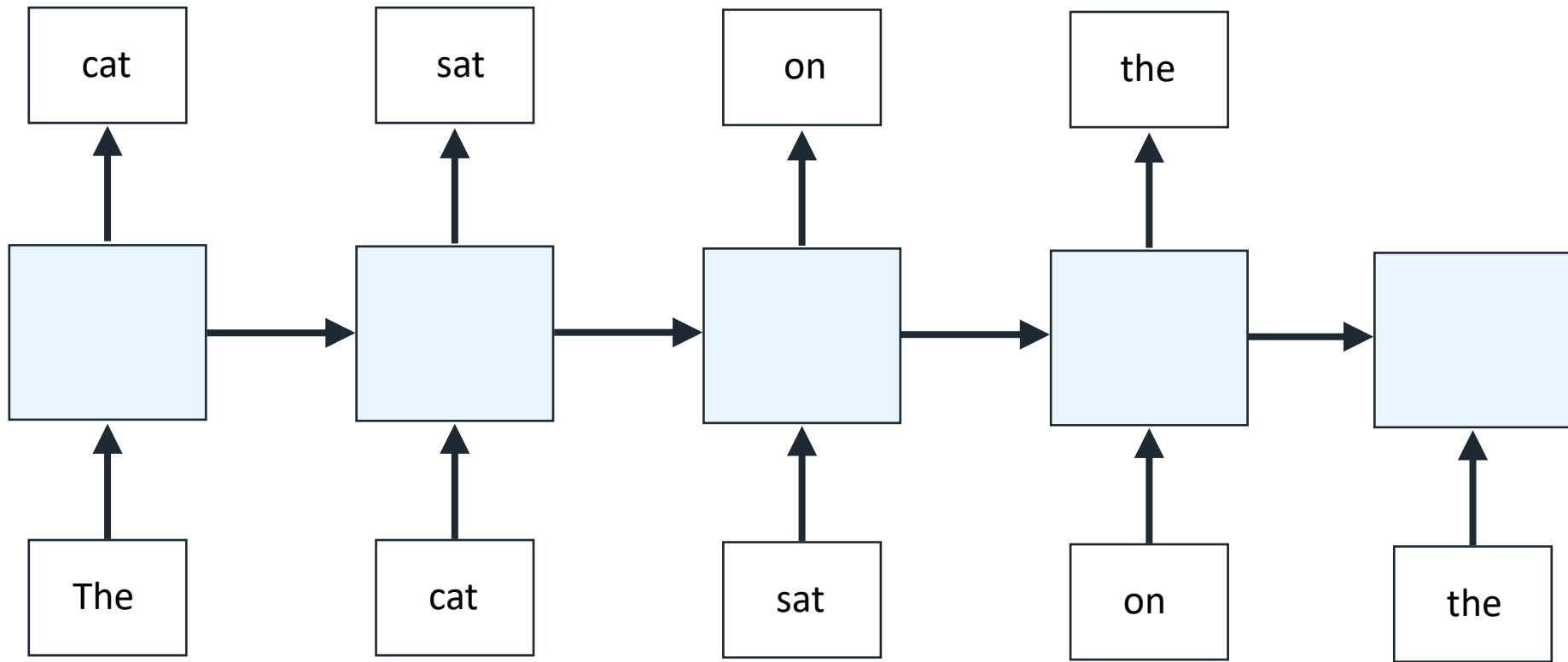
The recurrent network: a long computational graph

08 / 20



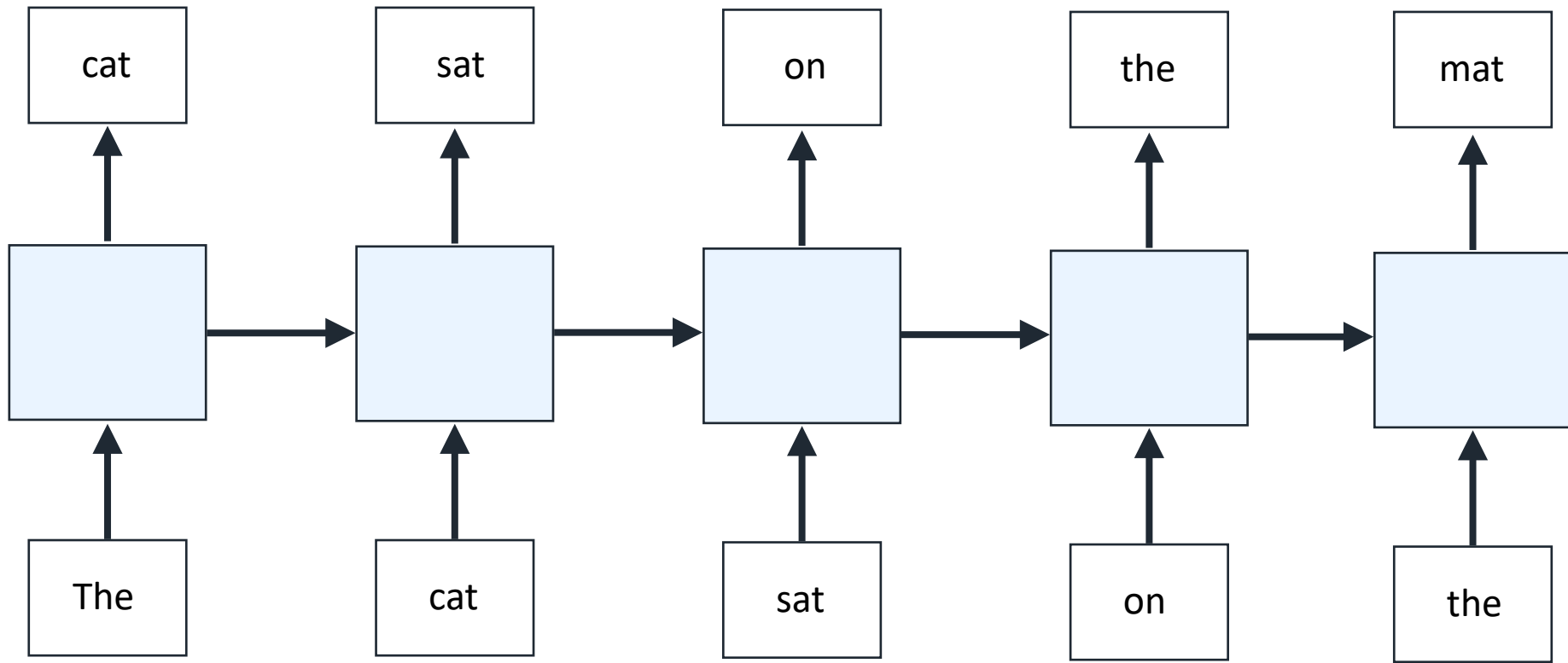
The recurrent network: a long computational graph

08 / 20



The recurrent network: a long computational graph

08 / 20



The unreasonable effectiveness of RNNs

09 / 20

- Train a character-level RNN with one job: predict the next character.
- It learns spelling, grammar, even LaTeX and code with matching brackets. (Karpathy, 2015)
- Pure next-token prediction is enough for the model to discover structure.
- Its failures are long-range: keeping track of things as sequences get longer.
- Astronomy: predict the next flux, and it learns the light curve's variability.

Key references: A. Karpathy: The Unreasonable Effectiveness of RNNs: <https://karpathy.github.io/2015/05/21/rnn-effectiveness/>

Why recurrence struggles

10 / 20

$$\nabla_{h_1} L = J_2^\top J_3^\top \dots J_T^\top \nabla_{h_T} L, \quad J_t = \frac{\partial h_t}{\partial h_{t-1}}$$

- A product of many Jacobians
- Gradients vanish (or explode!). Architectures like LSTMs can mediate this, but do not remove the long path.
- Limitation: the **whole past is squeezed into one fixed vector** h_t .
- Sequential: h_t needs h_{t-1} so time steps cannot be fully parallelized.

Why are RNNs not enough?

11 / 20

Method	Strengths	Limitations
word2vec	simple, intuitive embeddings (easy to understand via geometry)	order ignored, static, not context-aware
RNN	order matters, context-aware	Long graph, slow

We want a model where we can be aware of the context + parallel + not limited to a fixed-vector.

Attention: connecting our next prediction and the past 12 / 20

- Bahdanau et al., 2014, on machine translation.
- The idea: let each token look directly at all the others,
- weighting them by relevance

$$\mathbf{z}_i = \sum_j \alpha_{ij} \mathbf{v}_j$$

New representation

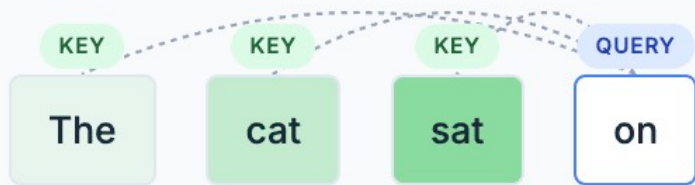
other tokens'
information

$$\hat{v}(x_Q) = \sum_{i=1}^n A_{Qi} v_i, \quad A_{Qi} \geq 0, \quad \sum_{i=1}^n A_{Qi} = 1$$

- To estimate a value at a new point x_Q , average the known values v_i .
- The weights are larger for points x_i that are more similar to x_Q .
- Prediction can be thought as a weighted average over examples.

The mechanics of attention

14 / 20



For each token position, we create three vectors:

- **Query:** What information am I looking for?

$$q_i = x_i W_Q$$

- **Key:** What info do I contain?

$$k_j = x_j W_K$$

- **Value:** What info do I pass on?

$$v_j = x_j W_V$$

$$\text{attn} = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)$$

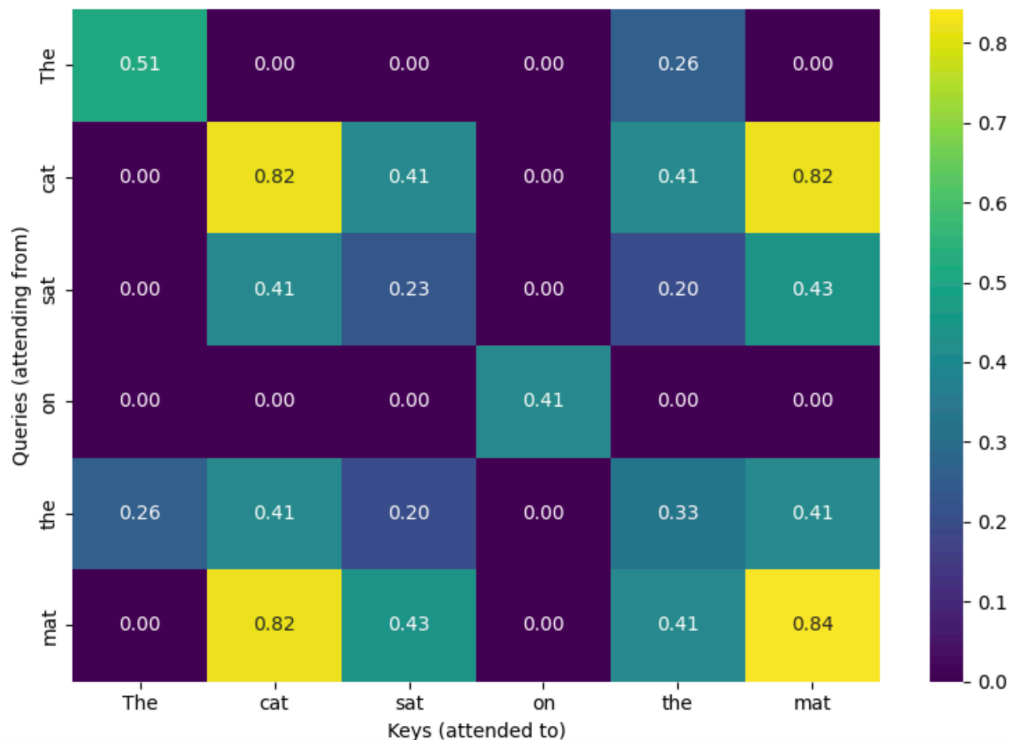
Why do we need $\sqrt{d_k}$?

The function of an attention head is to **move information between token positions**.

- This can be decomposed into two computational circuits that work together
 - The **QK Circuit** or "Where to Look" determines **which** tokens attend to a given query token (for all queries)
 - The **OV Circuit** or "What to Move" determines **what information** gets moved from these positions

What an attention pattern looks like

16 / 20



- Each query row shows how it weights the other tokens.
- Soft, content-based retrieval, rather than fixed positions.
- “cat” now depends on the sentence: the context-awareness.
- Static embeddings are now solved.

Several heads, several relations learnt

17 / 20

$$\text{head}^h = \text{Attention}(XW_Q^h, XW_K^h, XW_V^h)$$

- One head learns one particular relation.
- By running several in parallel, we can capture multiple, different relations.
- Concatenate the heads, then mix them with an output projection.
- For light curves, different heads can attend to cadence, color evolution, or local shape.

How about the position of the tokens?

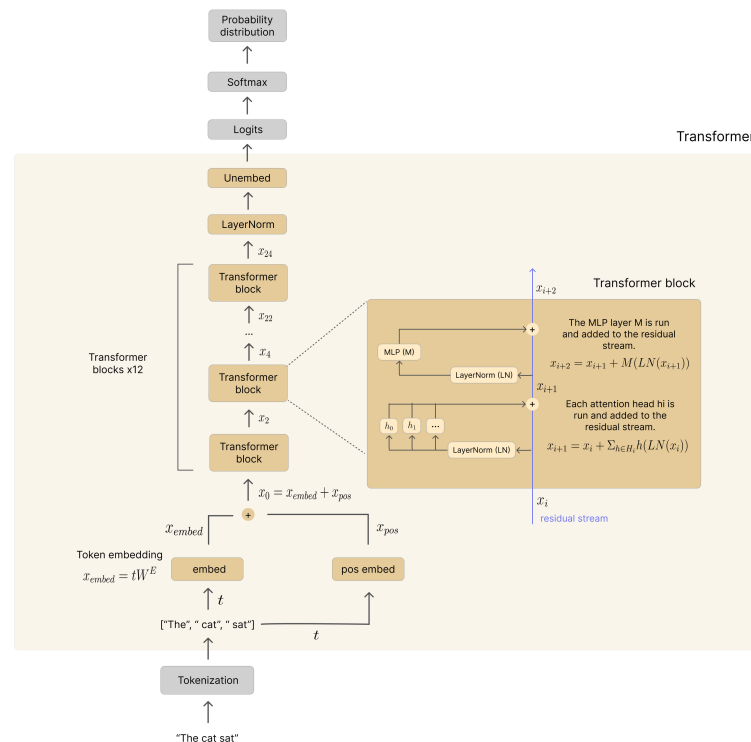
18 / 20

- Attention is permutation-invariant, with no sense of order.
- So we add position explicitly. Sometimes referred to as positional encoding.
- Position can be learned or hardcoded.
- For light curves, we can encode things like observation times.

The transformer block: a short computational graph

19 / 20

- Attention moves information between positions.
- A feed-forward layer processes each position.
- A residual stream is the space where each layer reads and writes & contextual information gets added.
- Every position reaches every other in one step. Process is parallel, with stable gradients.



- Tokens → static embeddings → RNN (context, but some limitations) → attention → transformer.
- Key idea: attention is similar to weighted averaging with a learned similarity kernel (Nadaraya–Watson).
- Exercise: implement the attention mechanism (notebook).
- Before Lecture 2, read “Attention Is All You Need” & and share your thoughts with your peers!
- Lecture 2: scaling these models up.